
Performance Engineering SLA, Analysis, and RAG Runbook

Standards, data dictionary, agent contracts, analysis rules, ingestion guidance, and evaluation criteria

Corpus ID	PTA-STANDARD-RAG-V2
Version	2.0 - Expanded RAG Edition
Source dataset	performance_test_dataset_1500_unique_names.csv
Rows represented	1,500
Primary consumer	Analysis Agent inside the Performance Test Agent

Document purpose

- Define how the Analysis Agent interprets current execution and monitoring outputs.
- Provide SLA and metric-band rules suitable for deterministic checks before LLM reasoning.
- Document handoffs among Orchestrator, Execution, Monitoring, Security, Analysis, and Report agents.
- Describe how the PDFs and the 1,500-row CSV should be ingested, filtered, retrieved, and evaluated.

Data handling note: This document uses aggregate metrics and selected Job IDs. Customer names and account values are intentionally excluded because they do not improve performance root-cause analysis. All content is synthetic and intended for RAG development, demonstrations, and testing.

Recommended RAG metadata

Field	Example value
document_type	Performance Engineering SLA, Analysis, and RAG Runbook
domain	banking-performance-testing
agent	analysis-agent
confidentiality	dummy-synthetic
chunk_strategy	section-aware, 500-900 tokens, 100-token overlap
retrieval_keys	scenario, API, status, symptom, bottleneck, recommendation, SLA

1. Dataset field dictionary and ingestion policy

Meaning, use, and handling of each structured field.

Field	Meaning	Analysis use	RAG handling
Job ID	Unique execution record	Citation and traceability	Store as exact metadata
Scenario	Business flow such as UPI or RTGS	Primary comparison group	Filter and embed
VU	Virtual-user identifier	Request-level trace only	Metadata; low semantic weight
API	Endpoint family	API-specific baseline	Filter and embed
Customer	Dummy person name	Not needed for performance RCA	Exclude or mask
From Account / To Account	Dummy account identifiers	Not needed for performance RCA	Exclude or mask
Amount	Transaction value	Optional workload segmentation	Store as numeric band
Request Time	Relative request timestamp	Trend and phase analysis	Normalize to elapsed seconds
Response Time (ms)	Request latency	Core SLA and similarity metric	Numeric metadata plus text band
HTTP Status	Functional outcome	Failure classification	Exact metadata
CPU (%)	Compute utilization	Bottleneck evidence	Numeric metadata plus band
Memory (%)	Memory utilization	Heap/pressure evidence	Numeric metadata plus band
DB Connections	Database concurrency	Pool/DB evidence	Numeric metadata plus band
Auth	Authentication result	Security cross-check	Exact metadata
SQL Injection	Security scan result	Security cross-check	Exact metadata
SLA	Pass/fail label	Ground-truth classification	Exact metadata
Bottleneck	Assigned failure category	RCA supervision and retrieval	Embed and exact metadata
Recommendation	Assigned action	Remediation retrieval	Embed; validate before applying

PII and security rule: ingestion should remove customer names, account numbers, secrets, tokens, authorization headers, and raw payloads unless a formally approved use case requires them. Performance analysis normally needs identifiers, metrics, timings, and technical context - not personal values.

2. SLA policy and metric bands

Deterministic rules should run before RAG retrieval and LLM interpretation.

Metric	Healthy	Warning	Critical	Interpretation
Response time	< 800 ms	800-1,500 ms	> 1,500 ms	Use scenario-specific percentile where available
P95 response	< 1,200 ms	1,200-2,000 ms	> 2,000 ms	Preferred service-level latency measure
Error rate	< 1%	1-3%	> 3%	Separate functional and infrastructure errors
CPU	< 75%	75-89%	>= 90%	Sustained critical CPU is stronger than a brief spike
Memory	< 75%	75-87%	>= 88%	Confirm with heap and GC evidence
DB connections	< 300	300-399	>= 400	Compare with configured maximum and active/wait state
HTTP status	200	intermittent 5xx	sustained 5xx	Status alone cannot identify root cause
Security	Auth PASS, SQL SAFE	isolated warning	security failure	Security Agent owns final security conclusion

Dataset-aware reference bands

For this synthetic dataset, median response time is 667 ms, P90 is 2778 ms, and P95 is 3464 ms. CPU P90 is 93%, memory P90 is 88%, and DB connection P90 is 417. These empirical values are retrieval features, not universal production SLAs.

SLA evaluation order

- Validate that required metrics exist and are numerically valid.
 - Apply explicit project SLA values when provided by the user or service contract.
 - Otherwise use scenario-specific baseline percentiles from approved historical data.
 - Use generic bands only as a fallback for demonstrations.
 - Report which SLA source was used so the decision is auditable.
-

3. Analysis Agent decision workflow

A staged process combining rules, statistics, retrieval, and grounded generation.

Step	Stage	Required behavior
A-01	Normalize inputs	Merge execution result, monitoring metrics, security result, environment metadata, and user question.
A-02	Validate quality	Detect missing values, impossible percentages, inconsistent timestamps, duplicate jobs, and unit mismatches.
A-03	Evaluate SLA	Calculate pass/fail, deltas from target, error rate, percentiles, and severity.
A-04	Identify dominant signals	Rank abnormalities: response, CPU, memory, DB, status, TPS, queues, dependency timing.
A-05	Create retrieval query	Build a structured query from scenario, API, status, metric bands, symptom terms, and question intent.
A-06	Retrieve evidence	Fetch historical cases, incident patterns, SLA rules, runbook sections, and counterexamples.
A-07	Generate hypotheses	Produce top causes with evidence for and against each cause.
A-08	Recommend checks	Ask for or inspect the metrics that would distinguish the hypotheses.
A-09	Recommend actions	Prioritize low-risk diagnosis and targeted remediation; explain validation.
A-10	Return citations	Include document IDs, section IDs, and historical Job IDs used.

Dominant-signal example

A current run has response 3,200 ms, HTTP 504, CPU 73%, memory 68%, and DB connections 470. The dominant abnormal signal is database concurrency, not CPU or memory. Retrieval should favor database pool and slow-query cases, while retaining network timeout as a secondary hypothesis because of HTTP 504.

4. Agent contracts and orchestration

Inputs and outputs should be explicit so the Analysis Agent receives complete evidence.

Agent	Required input	Expected output	Analysis dependency
Orchestrator	User question and context	Selected agents, execution mode, correlation ID	Must distinguish execution request from analysis-only request
Test Execution	Scenario, API, users, duration, script config	Raw results, percentiles, errors, logs, test metadata	Provides workload and measured behavior
Monitoring	Time-aligned infrastructure and application metrics	CPU, memory, DB, TPS, queues, alerts, timestamps	Provides bottleneck evidence
Security	Auth, authorization, rate limit, injection, token checks	Security status, findings, evidence	Prevents security errors being misclassified as performance
Analysis	All above plus RAG corpus	SLA verdict, hypotheses, evidence, recommendations, confidence, citations	Core reasoning component
Report	Analysis output and supporting artifacts	PDF/HTML/dashboard/executive summary/JSON	Must preserve evidence and uncertainty

Example orchestrator routing

User: "Why did yesterday's UPI performance test fail?"
Route: Monitoring Agent -> Analysis Agent -> Report Agent.
No new load test is executed. The Orchestrator passes the historical test ID or date range, monitoring artifacts, and the user question to the Analysis Agent.

Correlation requirements

- All agents must use the same correlation ID and execution ID.
- Metrics and logs must use a common timezone and synchronized timestamps.
- The Analysis Agent must know the workload phase: ramp-up, steady state, spike, or ramp-down.
- The Report Agent must not convert a low-confidence hypothesis into a definitive root cause.

5. RAG ingestion design for PDFs plus the 1,500-row CSV

Use structured analytics and unstructured retrieval together.

Recommended ingestion architecture

- Load the CSV into a structured store such as PostgreSQL, DuckDB, or a dataframe service for filtering, aggregation, and nearest-neighbor feature search.
- Convert selected CSV rows or grouped incidents into narrative documents only when semantic retrieval adds value.
- Chunk PDFs by section and knowledge entry, not arbitrary page boundaries.
- Attach metadata including document type, section ID, scenario, API, bottleneck, status, severity, and version.
- Use hybrid retrieval: metadata filters + BM25 keyword retrieval + vector similarity.
- Rerank the top candidates using the live metrics and question intent.
- Return citations and source excerpts to the Analysis Agent.

Chunking standards

Content type	Recommended unit	Target size	Overlap	Key metadata
Historical incident	One incident card	250-600 tokens	0-50	scenario, API, status, metrics, cause
RCA knowledge	One root-cause entry	300-700 tokens	50-100	cause, symptoms, evidence, remediation
SLA policy	One metric or policy section	200-500 tokens	50	metric, threshold, source, version
Agent contract	One agent or workflow stage	200-500 tokens	50	agent, input, output
CSV rows	Structured records; avoid embedding every raw row blindly	N/A	N/A	all numeric and categorical fields
CSV summaries	Scenario/bottleneck aggregates	200-600 tokens	50	group, time range, percentiles

6. Query construction, retrieval, and reranking

The query should encode measured evidence rather than only natural-language symptoms.

Structured retrieval object

```
{
  "intent": "root_cause_analysis",
  "filters": {"scenario": "UPI Transfer", "api": "UPI Transfer API", "sla": "FAIL"},
  "metrics": {
    "response_time_ms": {"value": 3200, "band": "critical"},
    "cpu_pct": {"value": 73, "band": "healthy"},
    "memory_pct": {"value": 68, "band": "healthy"},
    "db_connections": {"value": 470, "band": "critical"}
  },
  "http_status": 504,
  "semantic_terms": ["timeout", "database pool", "connection wait", "slow query"]
}
```

Reranking priorities

- Exact scenario and API match receives the highest metadata boost.
- Matching dominant metric band is more important than matching one generic word such as slow.
- A matching HTTP status is useful but should have lower weight than the resource pattern.
- Validated incidents should rank above unvalidated hypotheses.
- Recent incidents may rank higher when the application architecture changed, but stable foundational knowledge remains relevant.
- Include one disconfirming or alternative-cause chunk when confidence is not high.

Retrieval failure modes

- Embedding all 1,500 raw rows without metadata can return semantically similar but numerically irrelevant records.
 - Chunking an RCA entry across page boundaries can separate symptoms from remediation.
 - Stale SLA documents can make correct tests appear failed or failed tests appear healthy.
 - Duplicate documents can dominate top-k retrieval and falsely increase confidence.
 - Including customer names can create irrelevant similarity and unnecessary data exposure.
-

7. Analysis output schema and reporting rules

The output should be machine-readable and suitable for the Report Agent.

```
{
  "execution_id": "PT-2026-08-01-001",
  "overall_status": "FAIL",
  "sla_source": "project-sla-v3",
  "observations": [
    "response_time_ms=3200 exceeds target=800",
    "db_connections=470 is critical",
    "cpu_pct=73 and memory_pct=68 are not saturated"
  ],
  "hypotheses": [
    {"cause": "Database connection pool exhaustion", "confidence": 0.84},
    {"cause": "Slow database query", "confidence": 0.71},
    {"cause": "Network timeout", "confidence": 0.42}
  ],
  "recommended_checks": ["pool pending count", "acquisition time", "top slow SQL"],
  "recommended_actions": ["optimize slow SQL", "review pool capacity after DB headroom check"],
  "validation_test": "repeat same workload and compare pool wait, p95, error rate",
  "citations": ["DB-001", "DB-003", "JOB-20260801-xxxx"]
}
```

Report Agent rules

- Separate observations, hypotheses, recommendations, and validated conclusions.
 - Show confidence and evidence for each root-cause hypothesis.
 - Include before/after values when a fix has been validated.
 - Use an executive summary for leadership and a technical appendix for engineers.
 - Preserve source citations in PDF, HTML, dashboard, and JSON exports.
-

8. Evaluation plan for the RAG-enabled Analysis Agent

Measure retrieval quality, grounding, diagnostic usefulness, and safety.

Evaluation area	Metric	Target example	How to test
Retrieval	Incident Recall@5	≥ 0.85	Known questions with expected incident IDs
Retrieval	Relevant chunk precision	≥ 0.75	Human label top retrieved chunks
Grounding	Citation correctness	≥ 0.95	Verify every factual claim against cited chunk
Diagnosis	Top-1 cause accuracy	≥ 0.80 on labeled synthetic set	Compare with Bottleneck label and curated cases
Diagnosis	Top-2 cause recall	≥ 0.92	Allow plausible secondary cause
Recommendation	Action appropriateness	≥ 0.85 expert rating	Check whether action follows evidence and includes validation
Safety	Sensitive field leakage	0 occurrences	Prompt-injection and PII leakage tests
Robustness	Conflicting evidence handling	No unjustified high confidence	Provide contradictory retrieved cases
Latency	End-to-end analysis time	Project-specific	Measure filters, retrieval, reranking, generation

Suggested test questions

- Find historical UPI failures with high CPU but normal database connections.
 - Explain why a 504 response with 470 DB connections is more likely database-related than CPU-related.
 - Retrieve a counterexample where the same HTTP status had a different bottleneck.
 - Compare current IMPS response time with the IMPS historical baseline and P95.
 - Recommend checks before increasing JVM heap or database pool size.
 - Explain why customer names and account values should be excluded from performance RCA retrieval.
-

9. Operational governance and corpus lifecycle

Keep the RAG system accurate as tests, infrastructure, and SLAs evolve.

- Assign an owner and version to every SLA, runbook, and incident document.
- Re-index documents when metadata, chunking logic, or embedding model changes.
- Track which corpus version produced each Analysis Agent output.
- Expire or down-rank knowledge tied to retired architectures while preserving it for historical audits.
- Review recommendations that alter capacity, timeouts, memory, or security controls before automated execution.
- Log retrieved chunk IDs, reranking scores, final citations, and model output for debugging.
- Use role-based access control so users retrieve only documents they are authorized to view.
- Protect the RAG pipeline from prompt injection embedded in uploaded documents by treating document text as untrusted evidence, not instructions.
- Run regression evaluation after every corpus or prompt change.

Final implementation principle

The CSV should answer numerical questions such as counts, averages, percentiles, and nearest metric patterns. The PDFs should answer semantic questions such as known causes, diagnostic checks, remediation risks, orchestration rules, and reporting standards. The Analysis Agent becomes most useful when it combines both sources and cites the evidence used.

End of standards and RAG runbook.
